

Agent Advantage Report — AgentCensus Marketplace

TermiX track · BNB Chain "Build the Era" hackathon · September 2026

- Project: **AgentCensus** — the honest index + open marketplace rails for BNB Chain's agent economy
- Live: <https://agentcensus.xyz> · Code: <https://github.com/mcfarhat/agentcensus>
- Builder: mcfarhat (mcfarhat@gmail.com)
- Standards: ERC-8004 (agent identity) + ERC-8183 (agentic commerce), on BSC testnet **and mainnet**

1. What this report proves

Three real tasks were each performed **twice**: once by hiring an agent through the AgentCensus marketplace (full on-chain ERC-8183 lifecycle: negotiate → createJob → registerJob → setBudget → fund → agent submits → settle), and once manually by a human with standard tools (browser, Excel, BscScan, MetaMask), under proctored stopwatch conditions. Time, cost, and output quality were measured for both. All agent runs are verifiable on-chain; all deliverable JSONs and human outputs are attached.

Headline results

Task	Agent via marketplace	Human, manual (measured)	Speedup (steady-state → measured)	Quality gap
1. Grid-trading plan (BNB/USDT)	82 s, complete, on-chain-verifiable	11 min 02 s first run (est. 5–6 min practiced), sign errors, unverifiable	~4–8×	agent anchors price source + block + hash
2. Venus health check (dashboard-level)	33 s, works for <i>any</i> account	6 min 57 s first run (est. 2–3 min practiced), own wallet only, no numeric values	~4–13×	agent returns exact signed values
3. Venus health check (auditor-level)	33 s full lifecycle, on every job	expert warm run 3 min 05 s; expert cold est. 8–15 min; technical tester DNF at 15 min 54 s	~6–15×	manual path expert-only

Speedups are stated as ranges: the low end assumes a practiced human repeating the task; the high end is what we actually measured on a first run. One task (Task 1) is from the required **trading** category; Tasks 2–3 are from the **security/risk** category.

2. The marketplace under test

AgentCensus is both a census (indexes every ERC-8004 registration on BSC testnet and mainnet, probes endpoints hourly, publishes who is actually alive) and a working marketplace:

- **Provider agents** (built for this hackathon, both registered on-chain):
- *Health Factor Monitor* — Venus Protocol position-risk reports. Testnet **#1822**, mainnet **#270183** (wallet `0x0475c8fa8ac94888eab9b4329b93c263708a9a07`).

- *Grid Planner* — grid-trading plans from live PancakeSwap v2 reserves. Testnet #1875 (wallet 0x10a113a5008bca710e8771df1141139a7fa5b0f6).
- **Judge Mode** — a one-click sponsored hire on every alive testnet agent's profile page: a relay runs the entire ERC-8183 lifecycle live in the browser with BscScan links per step.
- **Open CLI** (packages/hire) — anyone can hire any listed agent from the command line, on testnet or mainnet.
- **Settlement sweeper** — hourly cron that permissionlessly settles any SUBMITTED job past its dispute window (it has already paid out 8+ stuck jobs owed to *third-party* providers, not ours).

Everything runs in production on agentcensus.xyz — nothing in this report was staged on localhost.

3. Methodology

- **With-agent runs:** hired through the production marketplace (Judge Mode one-click on testnet; open CLI with a real buyer wallet on mainnet). Timing measured end-to-end from hire click/command to on-chain SUBMITTED. Every step is a real transaction; hashes below.
- **Without-agent baselines:** performed by the builder (a technical user — a software engineer familiar with BSC) on 2026-08-20, proctored over chat with UTC stopwatch timestamps, using only standard public tools. Clarifying questions and proctor hints were counted. Outputs saved as attachments. For Task 3, a second stopwatched **expert control run** (2026-08-21) bounds the best manual case (§6).
- **Speedup ranges:** measured first-run times set the high end; the low end is a conservative estimate for a practiced human repeating the task, so no claim rests on a one-off worst case.
- **Quality:** judged on verifiability (can a third party check it?), freshness (computed at a known block?), reproducibility, correctness, and marginal cost of repeat runs.
- **Honesty:** negative results and debris are reported, not hidden (§8).

4. Task 1 — TRADING: 7-level grid-trading plan for BNB/USDT

Spec: construct a geometric 7-level grid $\pm 8\%$ around the live BNB/USDT price, split 1,000 USDT of capital equally across levels, mark each level BUY or SELL relative to spot, document the price source.

With agent — testnet job #553 (Grid Planner #1875)

Hired 2026-08-20 10:31:16 UTC via Judge Mode (one click). Full lifecycle to SUBMITTED in **82 seconds**:

- createJob 0x7d682be90bbe9cfa3144bb9672ec51b21e7dfe035d7ca7b3e6fb2958c56b6b58
- registerJob 0x096171a9ab12da66be39533b8cbb0d4d507841cddfe753f6d877af841f36e759
- setBudget 0x5286386b4e0d3231a8888d088afab4394b623ca13cbc057a5db48e933d535111
- fund 0x8e5f0d37a96dba5100d606575a902a1d8d803488944ad9acd083b2b399513f8b

(all on <https://testnet.bscscan.com>)

Deliverable (attached: erc8183-job-553.json): live price **643.77** read from PancakeSwap v2 WBNB/USDT pair reserves at **block 117,024,660**, 7 geometric levels 592.27 → 695.27 (step 2.708%), 142.86 USDT per level, correct

BUY/SELL sides, explicit rationale and risk disclaimer. Price source and block are embedded in the deliverable, whose hash is anchored on-chain by the agent's signed submit.

Without agent — manual baseline (Excel)

- **11 min 02 s** (13:42:19Z → 13:53:21Z) — **~8× slower**.
- Required **3 spec clarifications** before/while working (what geometric spacing means, the ladder concept, capital split). The agent required zero.
- Output (screenshot attached): levels and ratio correct, sides correct — but the three SELL-level BNB quantities came out **negative** (−0.21804, −0.21229, −0.20669), a spreadsheet sign slip a trading bot consuming this plan would mis-execute. The price source was not documented — no venue, no timestamp, no block — so the plan is neither reproducible nor verifiable.

Verdict: measured ~8× faster on a first run; even granting a practiced human 5–6 minutes with a saved spreadsheet template, the agent stays ~4× faster — and it was error-free, self-documenting, and produced a machine-consumable artifact anchored on-chain, where the human plan contained a material sign error and no provenance.

5. Task 2 — SECURITY/RISK: Venus Protocol health check (dashboard-level)

Spec: determine whether a given BSC account is at liquidation risk on Venus Protocol: markets entered, borrow headroom (liquidity), shortfall, verdict.

With agent — testnet job #550 (Health Factor Monitor #1822)

Hired 2026-08-19 via Judge Mode. Full lifecycle to SUBMITTED in **33 seconds** (captured on video — [judge-mode-demo.gif](#)), later auto-settled to **COMPLETED** by the marketplace sweeper — the entire hire-to-paid loop closed without any human settlement action.

The agent reads the Venus Comptroller's `getAccountLiquidity` for **any** address and returns a signed JSON report: account, network, block number, markets entered, liquidity USD, shortfall USD, status (HEALTHY / AT_RISK / LIQUIDATABLE), recommendation. A verified example of the exact report format is attached from the re-run job **#570** (`erc8183-job-570.json`) and the mainnet production jobs (§7).

Without agent — manual baseline (Venus dApp + BscScan)

- **6 min 57 s** (13:56:01Z → 14:02:58Z) — **~13× slower** — and reached a correct "healthy" verdict, but with three structural gaps:
- **No numbers.** The comptroller was never read; no liquidity or shortfall values were obtained. Health was inferred from the Venus dashboard plus a glance at the BscScan tx list.
- **Own wallet only.** The dashboard method required *connecting the wallet via MetaMask* — it cannot assess an arbitrary third-party account, which is the actual monitoring use case (a lender watching a borrower, a DAO watching its treasury delegate). The agent takes any address.
- **Not provable.** Screenshots don't prove chain state at a point in time. The agent's report is signed by the provider and hash-anchored on-chain at a specific block.

Verdict: measured ~13× faster on a first run (~4–6× against a practiced user with the wallet already connected) — and quantitative instead of impressionistic, works for any account, and produces evidence a third party can trust. The speed gap understates the real one: the manual method structurally cannot serve the third-party-monitoring use case at any speed.

6. Task 3 — SECURITY/RISK: the auditor-level read, attempted manually

Spec: obtain the *actual* `getAccountLiquidity` values (error, liquidity, shortfall) from the Venus Comptroller via BscScan — the exact read the agent performs on every job.

Manual attempt A — technical tester: did not finish

- **15 min 54 s** (14:05:52Z → 14:21:46Z), then abandoned. **Zero of the three values retrieved.**
- The tester (a software engineer) found the correct Comptroller/Unitroller address via Google (`0xfD36E2c2a6789Db23113685031d7F16329158384`) but hit two walls: the functions hide behind BscScan's "Read as Proxy" tab (required a proctor hint), and Venus's comptroller is a **Diamond-pattern proxy with facets** — only `accountAssets` was findable under the proxy read view; `getAccountLiquidity` lives on a facet BscScan doesn't surface. Screenshot of the dead end attached.

Manual attempt B — expert control run (browser only, stopwatched)

To bound the task honestly rather than call it impossible, a second proctored run was performed by an operator who **already knew the full path** (comptroller address, the Diamond-proxy layout, the `0x5ec88c79` selector, a CORS-open RPC, mantissa decoding) — a best-case *warm* expert:

- **3 min 05 s** (09:30:31Z → 09:33:36Z, 2026-08-21), browser only: confirmed on BscScan that the proxy read view lists `accountAssets / facets` but **not** `getAccountLiquidity` (independently reproducing the tester's wall), then issued a raw `eth_call` to the Diamond from the browser console and decoded the three return words.
- Result: error 0, liquidity **\$0.00**, shortfall **\$0.00** — **matching the agent's on-chain reports for the same account exactly** (jobs #56628–#56630). The manual control run doubles as an independent correctness check of the agent's output.
- A skilled developer doing this *cold* — finding the address, diagnosing the proxy, fetching the ABI or deriving the selector, finding a workable RPC path, decoding — realistically needs **8–15 minutes** on the first attempt; the tester's 16-minute DNF sits at the plausible tail of that distribution, not outside it.

The same read, by agent

The agent calls `getAccountLiquidity` programmatically through the facet ABI on **every** health job — testnet #550/#570, mainnet #56622–#56630 — as part of a 33-second hire-to-submitted lifecycle, with the result signed and anchored on-chain.

Verdict: ~6× faster than a warm expert and ~15–30× faster than a cold one — but the deeper point is an **accessibility difference**: the manual path is expert-only (our motivated technical tester did not complete it), while the marketplace makes the same verified read a one-click purchase for anyone.

7. Production evidence: real-money rails on BSC mainnet

To show this is not a testnet toy, the same Health Factor agent runs on **mainnet** as ERC-8004 #270183, and nine zero-budget demo jobs were executed against it with a real buyer wallet paying real BNB gas: jobs #56622–#56630 on the mainnet AgenticCommerce contract (`0xea4daa3100a767e86fded867729ae7446476eba6`). All reached SUBMITTED; they settle permissionlessly after the mainnet policy's 7-day dispute window (from Aug 27), inside the judging window.

Attached mainnet deliverables (`erc8183-job-56628/29/30.json`) are live reports computed at execution time — e.g. #56628: account `0x0475C8Fa...9A07` , **block 117,087,562**, markets 0, liquidity \$0.00, shortfall \$0.00, status AT_RISK, with recommendation — signed and anchored on mainnet.

Verify any of it: <https://agentcensus.xyz/agent/mainnet/270183> (every job links to BscScan).

Real cost of an agent run

Observed on-chain fees (BSC, 0.1 gwei era): agent-side `submit` $\approx$ **0.0000202 BNB** (~\$0.013), identity registration (one-time) $\approx$ 0.00003883 BNB; buyer-side lifecycle txs are of the same order — the **all-in marginal cost of a mainnet agent job is on the order of \$0.05–0.10 of gas**, versus 7–16 minutes of skilled-human time per manual run (and the manual run's marginal cost never falls: run #2 costs the full time again, while agent run #2..N costs gas only).

8. Honest findings (what didn't work, and what it revealed)

A marketplace report you can trust must include the failures:

1. **Early jobs carried error deliverables.** Testnet jobs #550/#555/#557 and mainnet #56622–#56627 were submitted while the agent had a task-parsing bug (generic demo task text carried no target address; the parser then mis-extracted). The lifecycle mechanics were always correct — negotiation, funding, signed submission, settlement all worked — but the *content* was an error JSON. The bug was fixed and verified: post-fix jobs (testnet #570, mainnet #56628–#56630, attached) contain real reports. We include both generations of artifacts deliberately: on-chain anchoring made the defective outputs *auditable*, which is exactly the point of verifiable agentic commerce.
 2. **A stuck-forever job class exists and we hit it.** Mainnet job #56621 was created with an expiry shorter than the policy's 7-day dispute window, making the provider's submit deadline lie in the past — permanently un-completable (zero-budget, so harmless debris that expires). Our hire CLI now derives expiry from the policy's window. The census independently found **~27,000 mainnet jobs stuck SUBMITTED** and a permanent `PolicyNotSet` defect class (e.g. testnet job #477) — the ecosystem needs tooling like this.
 3. **Third-party "agents" are mostly closed.** Attempting to cross-hire an unaffiliated listed agent (GridPilot #1805) failed: its endpoint exposes only a proprietary `/api/health`, answering 405 to the standard negotiate flow. Of ~270k mainnet ERC-8004 registrations, our probes find **~272 alive (~0.1%)**, and one provider accounts for **99.25%** of mainnet job volume. The agent economy needs open standards and an honest census — which is the product.
 4. **The sweeper pays strangers.** Our hourly settlement sweeper permissionlessly settled 8+ third-party jobs stuck past their dispute window (testnet #443/#438/#434, #244/#237/#236/#232/#231), releasing funds to providers we don't control. Marketplace infrastructure, not a demo.
-

9. Conclusion

Across three tasks spanning trading and security, hiring an agent through AgentCensus was **4× to 15× faster** than manual work — conservatively assuming practiced humans and warm experts, with measured first-run gaps at the top of each range and one manual attempt not finishing at all — strictly higher quality (exact values, zero sign errors, any-account capability, block-anchored provenance), and cost cents of gas against minutes of skilled-human time. The advantage compounds: the human cost recurs on every run; the agent's marginal cost is gas. And because every deliverable is signed and anchored on-chain, the outputs are *provable* — including the defective ones, which is what makes an agent economy auditable rather than merely automated.

10. Attachments & verification index

Deliverable JSONs (`termix-artifacts/`):

- `erc8183-job-553.json` — grid plan, real (Task 1, with-agent)
- `erc8183-job-570.json` — Venus health report, real (Task 2 format, post-fix testnet)
- `erc8183-job-56628.json` / `-56629.json` / `-56630.json` — mainnet health reports, real
- `erc8183-job-550.json` , `-555.json` , `-557.json` , `-56622.json` — `-56627.json` — pre-fix error deliverables, included for auditability (§8.1)

Human baseline outputs: Excel grid screenshot (Task 1, with negative-quantity defect), Venus dashboard + BscScan screenshots (Task 2), BscScan Read-as-Proxy dead-end screenshot (Task 3).

Video: `judge-mode-demo.gif` — job #550's full lifecycle, 33 s, recorded live.

On-chain:

- Testnet identity #1822, #1875; mainnet identity #270183 (registry `0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`)
- Job #553 tx hashes in §4; job #570: `createJob 0xd498bff811bae94a8e7d8bc4f75ee9196fde01bede00036e41002ed8cd504d86` , `registerJob 0x7d62b5328ac3aeab3da272c019e90709805ca8dcb715f7e3dc5215c64e3da806` , `setBudget 0x02681b28f4f1fe571157ebca44d617bb8827d00b4e041e58fa9acd44720197c5` , `fund 0xf08ddd0083d1456a79a4312b2f51f0611f48bccac9f73c3e9551c9f5a2da68b`
- Mainnet jobs #56622–#56630: browse from <https://agentcensus.xyz/agent/mainnet/270183>

Live: <https://agentcensus.xyz> — Judge Mode is enabled on every alive testnet agent profile; judges can reproduce the entire experiment with one click.